

Einführung in Data Science

Datengetriebene Modelle



Datengetriebene Modelle

Inhalte

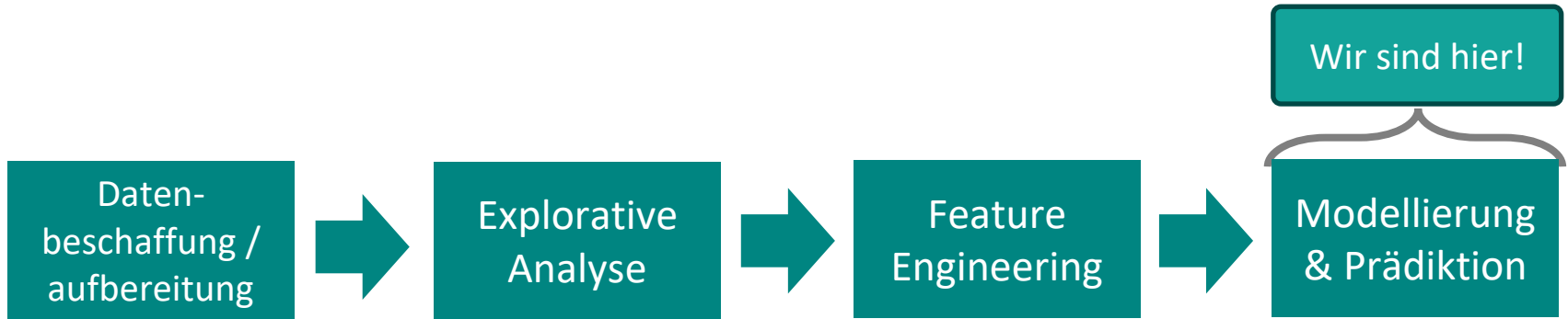
1. Überblick & Grundbegriffe
2. Einführung in Python (NumPy und Pandas)
3. Eindimensionale EDA und Visualisierungen
4. Visualisierungen und mehrdimensionale EDA
5. Mehrdimensionale EDA
6. Dimensionsreduktion: PCA
7. Dimensionsreduktion: MDS, Isomap
8. Clustering: Algorithmen
9. Clustering: Validierung
10. Probeklausur
11. Feature Engineering, datengetriebene Modelle
- ➡ 12. Datengetriebene Modelle
13. Zeitreihen

Überblick &
Grundlagen

Explorative
(Daten-)
Analyse (EDA)

Feature
Engineering &
Modellierung

Datengetriebene Modelle



Heute

1. Wiederholung
2. Modellierung

1. Wiederholung

1. Wiederholung

Feature Engineering

- Transformation von Daten
 - Allgemeine Methoden
 - Expertenwissen
 - Repräsentation ist entscheidend für Modellierung
 - Oft werden neue Features aus Daten erzeugt (→ Feature Engineering)
 - Feature Engineering „vermittelt“ zwischen Daten und Modellen
 - Häufig zeitaufwändig, domänenspezifisch und entscheidend für Erfolg/Misserfolg
-
- Explorative Datenanalyse: Kennenlernen der Daten und Entwicklung von Fragen
 - Feature Engineering: Daten für die daten-getriebene Modellierung transformieren

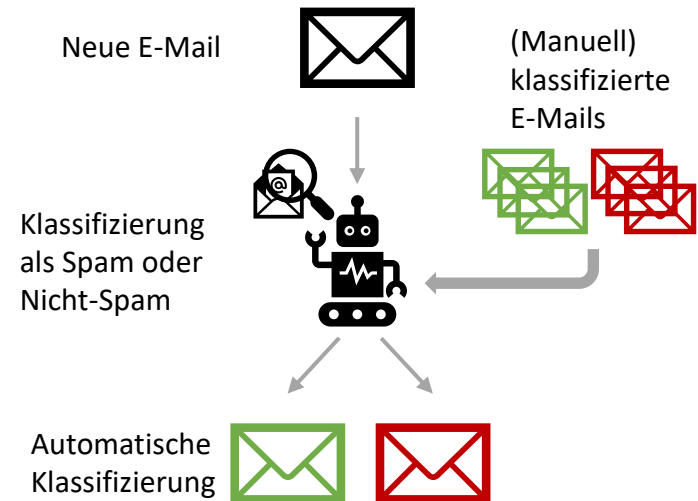
1. Wiederholung

Wann ist maschinelles Lernen hilfreich?

- Es gibt Muster in den Daten
- Wir können diese Muster nicht (effizient) mathematisch beschreiben
- Wir haben Daten

Supervised Learning (überwachtes Lernen)

- Input: $x \in X$ (Features)
- Output: $y \in Y$ (Label)
- (Unbekannte) Zielfunktion: $f(x) = y$
- Datensatz/Stichprobe: $(x_1, y_1), \dots, (x_n, y_n)$
- Ziel: Approximation von f basierend auf den Daten
 - Finde g sodass $g \approx f$



1. Wiederholung

Beurteilung eines Klassifikators

		Vorhergesagte Klasse	
		Positiv	Negativ
Wahre Klasse	Positiv	True Positive (TP)	False Negative (FN)
	Negativ	False Positive (FP)	True Negative (TN)

➤ Was ist schlimmer? *FP* oder *FN*?

Smartphone Entsperren mit Fingerabdruck

		Vorhergesagte Klasse	
		Pos.	Neg.
Wahre Klasse	Pos.	TP: Akzeptiere Fingerabdruck des Besitzers	FN: Akzeptiere Fingerabdruck des Besitzers nicht
	Neg.	FP: Akzeptiere Fingerabdruck eines Angreifers	TN: Akzeptiere Fingerabdruck eines Angreifers nicht

FBI: Terroristenerkennung

		Vorhergesagte Klasse	
		Pos.	Neg.
Wahre Klasse	Pos.	TP: Erkenne Terrorist	FN: Identifiziere Terrorist als Zivilist
	Neg.	FP: Identifiziere Zivilist als Terrorist	TN: Erkenne Zivilist

➤ Es kommt darauf an!

1. Wiederholung

Beurteilung eines Klassifikators

Es gibt eine Vielzahl von Gütemaßen

$$\text{➤ Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$$

$$\text{➤ Precision} = \frac{TP}{TP+FP}$$

$$\text{➤ Recall} = \frac{TP}{TP+FN} = \frac{TP}{P} \quad (\text{auch Sensitivität oder TPR – True Positive Rate})$$

$$\text{➤ Specificity} = \frac{TN}{TN+FP} = \frac{TN}{N} \quad (\text{auch Spezifität oder TNR – True Negative Rate})$$

$$\text{➤ } F_1 = 2 \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Alle diese Gütemaße fassen Informationen zusammen, nur die Konfusionsmatrix (confusion matrix) zeigt das gesamte Bild:

		Vorhersage	
		Pos.	Neg.
Wahre Klasse	Pos.	TP	FN
	Neg.	FP	TN

1. Wiederholung

k -nächste Nachbarn

Gegeben:

- Datensatz $(X_1, y_1), (X_2, y_2), \dots (X_n, y_n)$
- Anzahl Nachbarn k
- Input X

Gesucht:

- Zielgröße y



k -nearest neighbors (k -nächste Nachbarn Algorithmus)

1. Berechne für jeden Punkt $X_1, X_2, \dots, X_n$ den Abstand zu X
2. Suche aus dem Datensatz die k nächsten Punkte um X
3. Bestimme häufigste Klasse y der Nachbarn

2. (Datengetriebene) Modellierung

2. Modellierung

Supervised Learning – Beispiele

Iris (<https://www.kaggle.com/datasets/uciml/iris>)

- Input: Merkmale der Blüten (Kron- und Kelchblatt je Breite und Länge)
- Output: Art der Pflanze (iris setosa, iris versicolor, iris virginica)
- Datensatz: 150 Paare (Merkmale und Arten, je 50 pro Art)
- Ziel: Vorhersage der Pflanzenart basierend auf Merkmalen



Iris setosa



Iris versicolor



Iris virginica

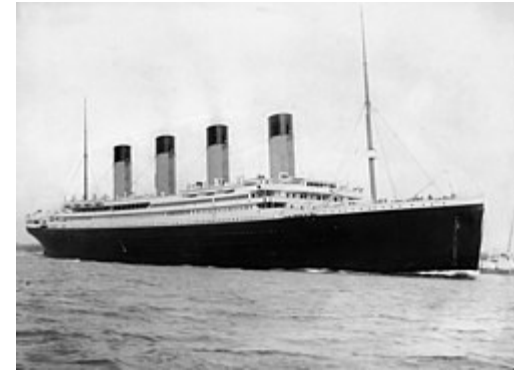
Quelle: Wikipedia

2. Modellierung

Supervised Learning – Beispiele

Titanic (<https://www.kaggle.com/c/titanic/>)

- Input: Infos über Passagiere (Klasse, Geschlecht, Alter, ...)
- Output: Überleben (ja/nein)
- Datensatz: 1309 Paare (Infos und Überleben)
- Ziel: Vorhersage, ob Passagier überlebt hat
- Standardbeispiel im maschinellen Lernen
 - Warum ist das keine nützliche ML Anwendung?



Quelle: Wikipedia

2. Modellierung

Supervised Learning – Beispiele

Housing Prices (<https://www.kaggle.com/harrywang/housing>)

- Input: Infos über Bezirk (Längen-/Breitengrad, Hausalter (Median), Anzahl Haushalte, ...)
- Output: Mittlerer (Median) Wert der Häuser in Bezirk
- Datensatz: 20640 Paare (Infos und Wert)
- Ziel: Vorhersage des mittleren Hauswerts in gegebenem Bezirk

- **Hinweis:** Ein Bezirk umfasst in der Regel eine Bevölkerung von 600 bis 3000 Menschen.

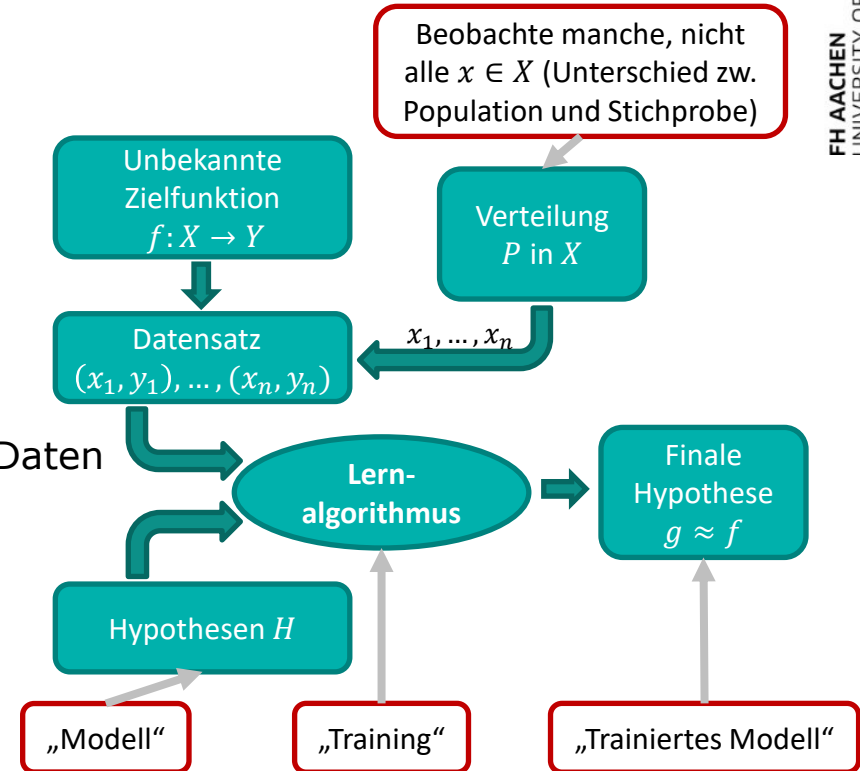


Quelle: Wikipedia

2. Modellierung

Supervised Learning – Diagramm

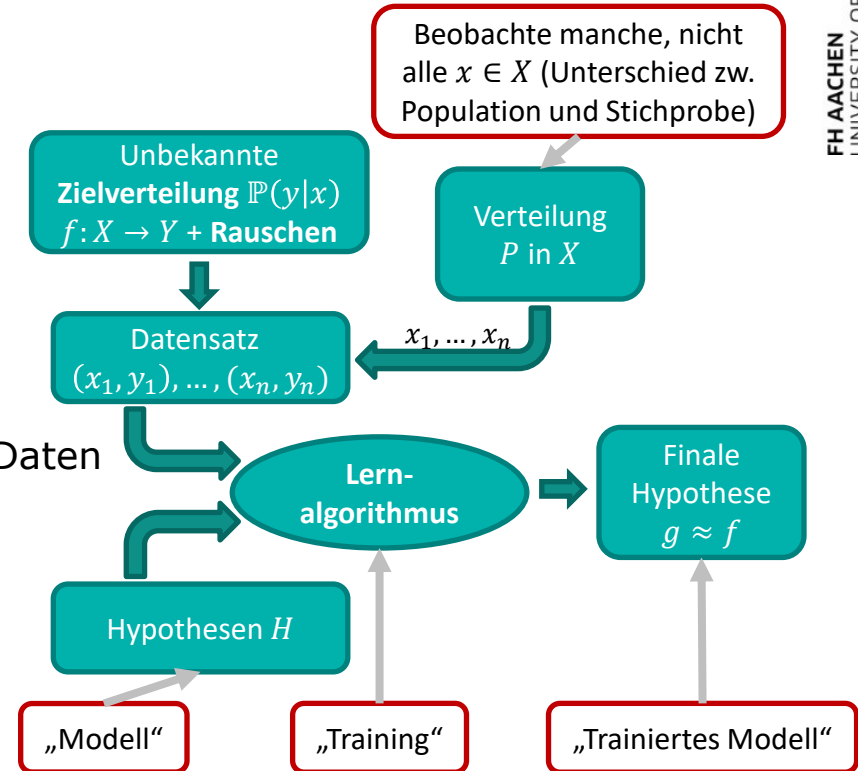
- Input: $x \in X$ (Features)
- Output: $y \in Y$ (Label)
- (Unbekannte) Zielfunktion: $f(x) = y$
- Datensatz/Stichprobe: $(x_1, y_1), \dots, (x_n, y_n)$
- Ziel: Approximation von f basierend auf den Daten
 - Finde g sodass $g \approx f$



2. Modellierung

Supervised Learning – Diagramm

- Input: $x \in X$ (Features)
- Output: $y \in Y$ (Label)
- (Unbekannte) Zielfunktion: $f(x) = y$
- Datensatz/Stichprobe: $(x_1, y_1), \dots, (x_n, y_n)$
- Ziel: Approximation von f basierend auf den Daten
 - Finde g sodass $g \approx f$



2. Modellierung

Supervised Learning – Beispiele

Rotwein Qualität (<https://www.kaggle.com/datasets/uciml/red-wine-quality-cortez-et-al-2009>)

- Input: Infos über Wein (Zitronensäure, Restzucker, Dichte, pH-Wert, Alkohol, ...)
- Output: Qualität des Weins
- Datensatz: Daten von 1599 Rotweinen
- Ziel: Vorhersage der Weinqualität anhand von Merkmalen

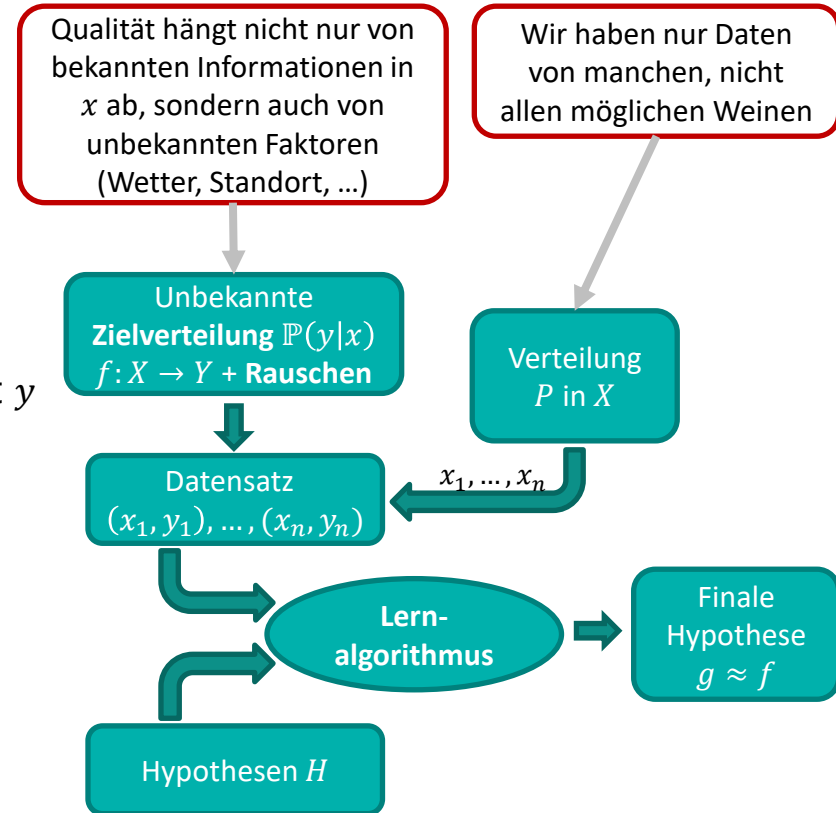


Quelle: Kaggle

2. Modellierung

Supervised Learning – Diagramm

- Input: $x = (\text{Säure, Zucker, Dichte, pH, alc., ...})$
- Output: $y = \text{Weinqualität}$
- Datensatz: $(x_1, y_1), \dots, (x_{1599}, y_{1599})$
- Ziel: Finde Funktion g , die die Weinqualität y für jedes x so gut wie möglich schätzt



2. Modellierung

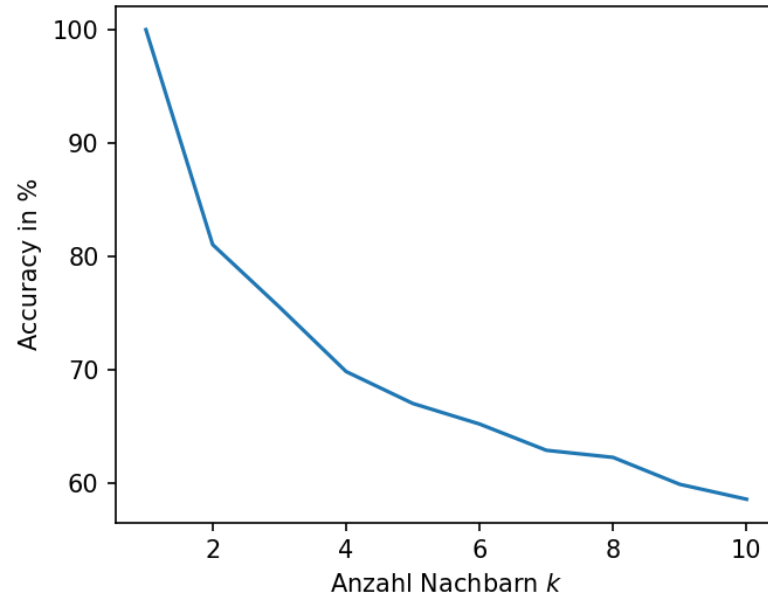
Vorhersage der Rotwein-Qualität mit k -nearest neighbors

```
import numpy as np
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier

if __name__ == '__main__':
    filepath = "../data/wine_quality.csv"
    df = pd.read_csv(filepath)
    X, y = df.iloc[:, :-1].to_numpy(), df.iloc[:, -1].to_numpy()

    accuracies = []

    for k in range(1, 11):
        model = KNeighborsClassifier(n_neighbors=k)
        model.fit(X, y)
        y_pred = model.predict(X)
        accuracy = np.sum(y_pred == y) / len(y) * 100
        accuracies.append(accuracy)
```



Wie sollten wir k wählen?

- $k = 1$ hat optimale Accuracy (100%)
- Ist das sinnvoll?

2. Modellierung

Vorhersage der Rotwein-Qualität mit k -nearest neighbors

- $k = 1$ hat optimale Accuracy (100%)
- Sollten wir also $k = 1$ wählen?
- Die Accuracy von 100% gibt an, wie gut Vorhersagen für **bekannte** Weine sind
- Was uns interessiert: Wie gut sind Vorhersagen für **neue** Weine

- Die Daten, die zum Training benutzt werden, heißen *Trainingsdaten*
- Die Accuracy basierend auf den Trainingsdaten heißt *in-sample accuracy*
- Was uns interessiert: Die (unbekannte) *out-of-sample accuracy* $\mathbb{P}(y = g(x))$
 - Die out-of-sample accuracy ist unbekannt
 - Wir müssen Sie schätzen
 - Die in-sample accuracy ist ein verzerrter Schätzer

2. Modellierung

Wie können wir die unbekannte *out-of-sample accuracy* $\mathbb{P}(y = g(x))$ schätzen?

- Die in-sample accuracy ist ein verzerrter Schätzer
- Stattdessen: Unterteile Daten/Stichprobe in Trainings- und Testdaten
- Oft: 80% Trainings- und 20% Testdaten

- Python

```
from sklearn.model_selection import train_test_split  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```



- Schätze out-of-sample Accuracy mit Testdaten
- Wenn der out-of-sample Fehler klein ist, *verallgemeinert* das Modell gut

2. Modellierung

Code in `topic12_modelling/
knn_exercise.py`

Vorhersage der Weinqualität – Übung

1. Laden Sie den Datensatz `data/wine_quality.csv` mit Pandas.
2. Unterteilen Sie mit der Methode `train_test_split` den Datensatz in 80% Trainings- und 20% Testdaten.

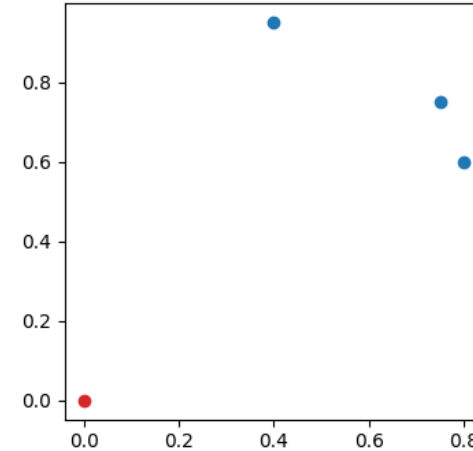
Um Vergleichbarkeit zu gewährleisten, übergeben Sie als `random_state` den Wert 24.

3. Erstellen Sie k NN-Modelle für $k = 1, \dots, 30$ basierend auf Ihren Trainingsdaten und evaluieren die Modelle anschließend mit den Testdaten.
 4. Welche Wahl von k verallgemeinert am besten?
-
5. Wie können Sie durch sinnvolles preprocessing die Vorhersage verbessern?

2. Modellierung

k-nearest neighbors

- Was sind die „nächsten“ Punkte?
- Welcher Punkt ist am nächsten am Ursprung (0,0)?
 - $x_1 = (0.4, 0.95)$
 - $x_2 = (0.75, 0.75)$
 - $x_3 = (0.8, 0.6)$
- Abhängig von der Metrik:
 - Manhattan-Metrik: $d_1(x, y) = \sum_{i=1}^d |x_i - y_i|$
 - Euklidische Metrik: $d_2(x, y) = \sqrt{\sum_{i=1}^d (x_i - y_i)^2}$
 - Maximum-Metrik: $d_\infty(x, y) = \max_{i=1, \dots, d} |x_i - y_i|$



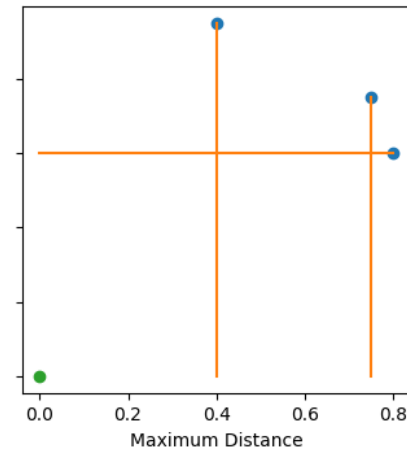
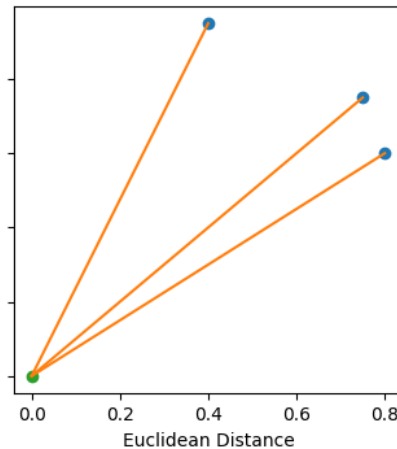
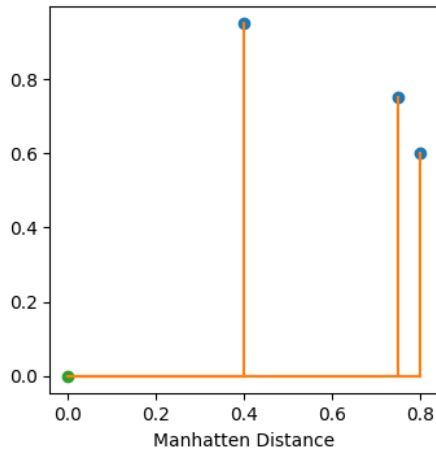
	d_1	d_2	d_∞
x_1	1.35	1.03	0.95
x_2	1.50	1.06	0.75
x_3	1.40	1.00	0.80

2. Modellierung

k-nearest neighbors

- Welcher Punkt ist am nächsten am Ursprung (0,0)?
- Abhängig von der Metrik
 - $x_1 = (0.4, 0.95)$
 - $x_2 = (0.75, 0.75)$
 - $x_3 = (0.8, 0.6)$

	d_1	d_2	d_∞
x_1	1.35	1.03	0.95
x_2	1.50	1.06	0.75
x_3	1.40	1.00	0.80



2. Modellierung

k-nearest neighbors

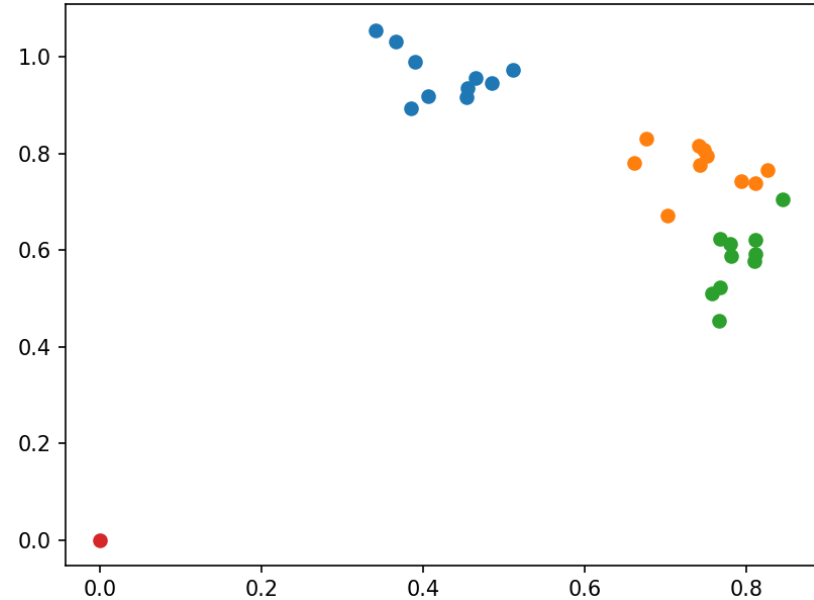
➤ Warum ist das wichtig?

```
import numpy as np
from sklearn.neighbors import KNeighborsClassifier

if __name__ == '__main__':
    means = np.array([[0.4, 0.95], [0.75, 0.75], [0.8, 0.6]])

    X = np.random.randn(3, 10, 2) * 0.05
    noise = np.expand_dims(means, axis=1)
    X = X + noise
    X = X.reshape(30, 2)
    y = np.array([0] * 10 + [1] * 10 + [2] * 10)
    origin = np.zeros((1, 2))

    for metric in ['manhattan', 'euclidean', 'infinity']:
        model = KNeighborsClassifier(metric=metric)
        model.fit(X, y)
        pred = model.predict(origin)
        print(f"Metric: {metric}, prediction: {pred}")
```



```
>>> Metric: manhattan, prediction: [0]
>>> Metric: euclidean, prediction: [2]
>>> Metric: infinity, prediction: [1]
```

2. Modellierung

***k*-nearest neighbors**

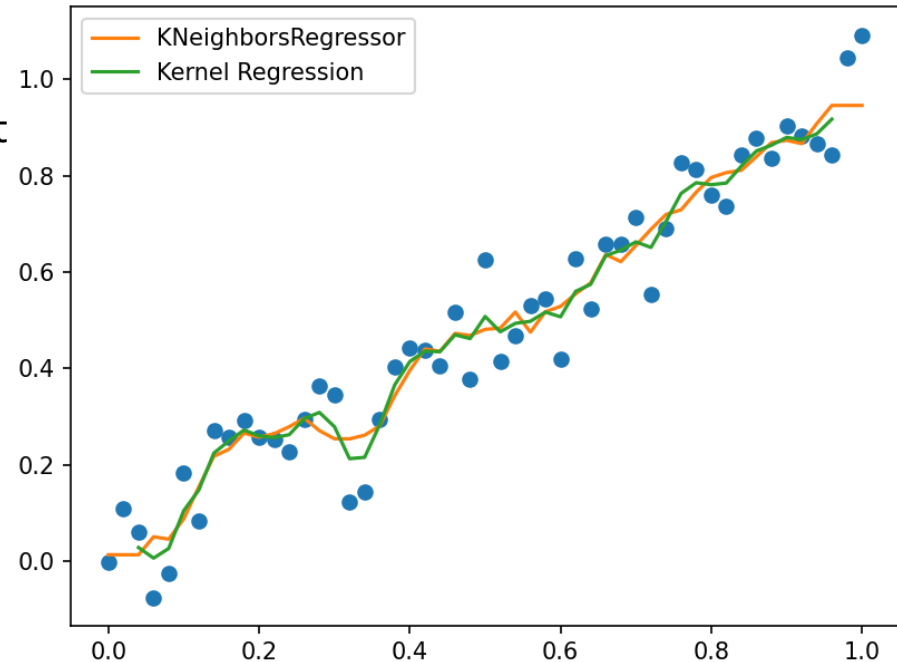
k-nearest-neighbors kann auch zur Regression benutzt werden

➤ Wie?

- Bilde Durchschnitt über Nachbarn
- Besser: Bilde gewichteten Durchschnitt
 - abhängig von Entfernung
 - Kernel Regression und Local Polynomial Regression

```
model1 = KNeighborsRegressor()
model1.fit(x_exp, y)
pred1 = model1.predict(x_exp)

kernel = np.array([0.1, 0.2, 0.4, 0.2, 0.1])
pred2 = np.convolve(y, kernel, mode='valid')
```



Datengetriebene Modelle

Inhalte

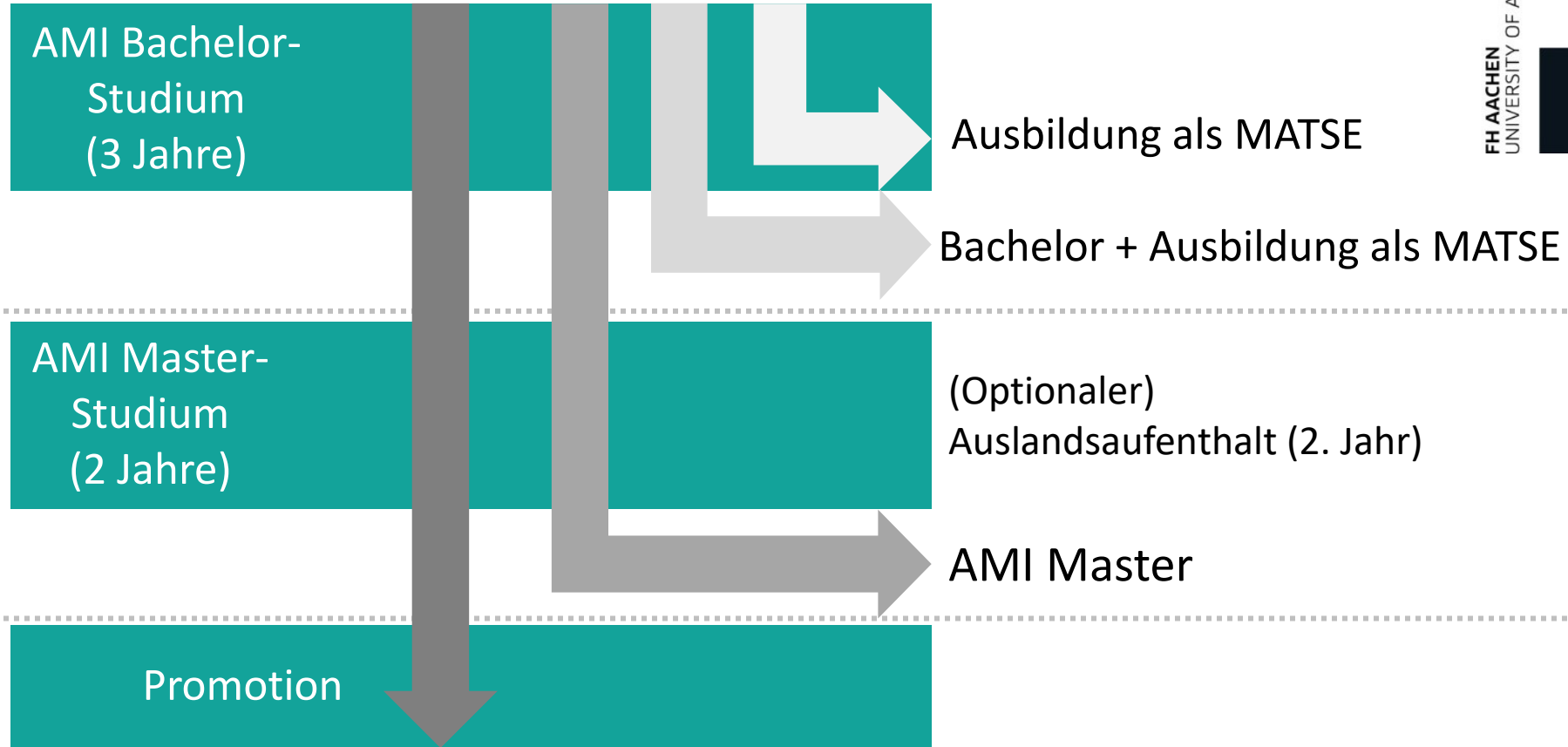
1. Überblick & Grundbegriffe
 2. Einführung in Python (NumPy und Pandas)
 3. Eindimensionale EDA und Visualisierungen
 4. Visualisierungen und mehrdimensionale EDA
 5. Mehrdimensionale EDA
 6. Dimensionsreduktion: PCA
 7. Dimensionsreduktion: MDS, Isomap
 8. Clustering: Algorithmen
 9. Clustering: Validierung
 10. Probeklausur
 11. Feature Engineering, datengetriebene Modelle
 12. Datengetriebene Modelle
- ➡ Und jetzt?

Überblick &
Grundlagen

Explorative
(Daten-)
Analyse (EDA)

Feature
Engineering &
Modellierung

Data Science & Statistik – FH Aachen



Data Science & Statistik – FH Aachen



Einführung Data Science

Machine Learning

Weitere Wahlpflichtmodule
(z.B. Data Analysis)

AMI Bachelor

AMI Master

Methoden-spezifische Vorlesungen

Machine Learning 1: Grundlagen
und statistische Lerntheorie

Machine Learning 2: Deep Learning

Viele weitere Wahlpflichtmodule
(z.B.: numerische Algorithmen,
Datenanalytik, statistische Modellierung, ...)

Datendomänen-spezifische Vorlesungen

Zeitreihen

Zeitreihen und Prognoseverfahren

Bilder

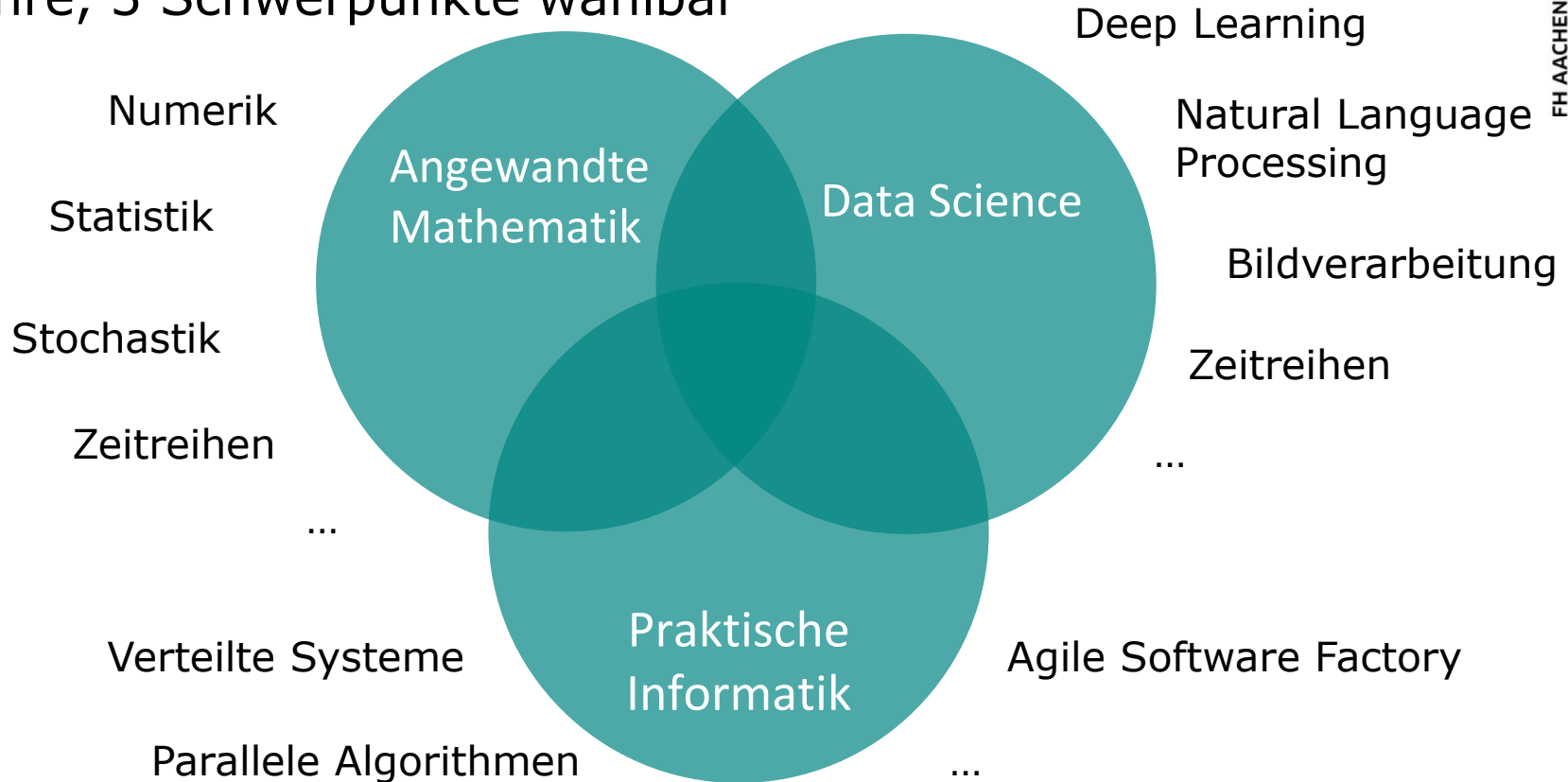
Digitale Bildverarbeitung

Sprache

Natural Language Processing

M.Sc. Angewandte Mathematik und Informatik

2 Jahre, 3 Schwerpunkte wählbar



Beispiele für Berufsfelder in Industrie oder Forschung

- Simulation und Optimierung
- Data Science & Machine Learning Engineering
- Softwareentwicklung
- Beratung
- Produktentwicklung
- Statistische Qualitätskontrolle
- ...

Weitere Informationen zum Masterstudiengang
finden Sie auf der Webseite der Hochschule:

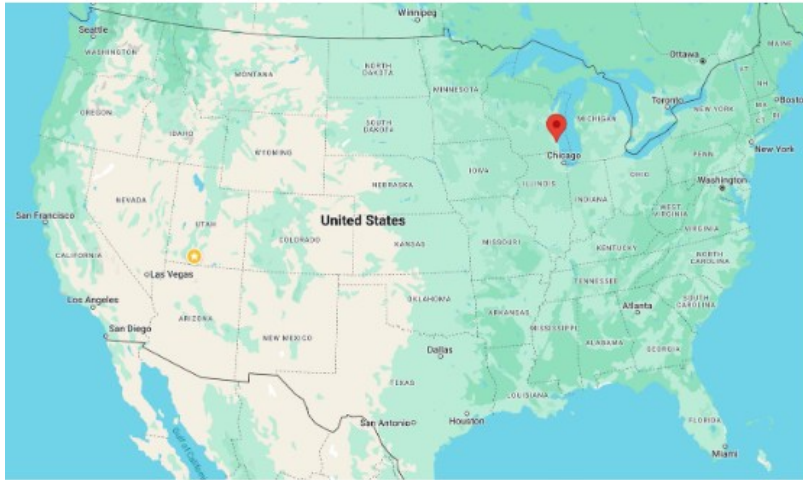


Antworten zu vielen praktischen
Studiumsfragen finden Sie auch in den „Häufig
gestellten Fragen“ (FAQ):



Möglichkeit eines **Dual Degree**

- Auslandsaufenthalt (2. Jahr)
- University of Wisconsin-Milwaukee
- 2 Abschlüsse: M.Sc. AMI und M.Sc. Mathematics



direkt am Lake Michigan
150km nördlich von Chicago



ca. 600 000 Einwohner
tiefe Winter, warme Sommer

Datengetriebene Modelle

Rückblick

- Wie ist das "Supervised Learning - Diagramm" zu interpretieren?
- Wie erhalten wir beim kNN-Algorithmus ein Modell mit 100% Accuracy?
- Wie sind in-sample und out-of-sample Accuracy definiert?
 - Was ist der Unterschied?
- Warum können wir nicht die in-sample Accuracy als Schätzer für die out-of-sample Accuracy nutzen?
- Wie können wir die out-of-sample Accuracy schätzen?
 - Warum wollen wir sie überhaupt schätzen?
- Wovon hängt es ab welche Punkte die nächsten Nachbarn sind?
- Wie kann mit dem k-nearest neighbors Algorithmus eine stetige Variable vorhergesagt werden?